Maharashtra Education Society's
Institute of Management and Career Courses (IMCC), Pune
AUTONOMOUS

Academic Year 2026-27
FY MCA- SEM I
IT12 Data Structure and Algorithms
Practice Assignment1

Roll No: 2601178
Name: Prasad Govindrao Shinde.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    # 1. Create linked list
    def create(self, values):
        for value in values:
            self.insert_end(value)

    def insert_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        current = self.head
        while current.next:
            current = current.next

        current.next = new_node

    # 2. Traverse and print
    def display(self):
        current = self.head
        while current:
            print(current.data, end=" ")
```

```
                current = current.next
    print()

# 3. Insert at a specific position (0-based)
def insert_at(self, data, position):
    new_node = Node(data)

    if position == 0:
        new_node.next = self.head
        self.head = new_node
        return

    current = self.head
    for _ in range(position - 1):
        if current is None:
            print("Invalid position")
            return
        current = current.next

    if current is None:
        print("Invalid position")
        return

    new_node.next = current.next
    current.next = new_node

# 4. Find middle node
def find_middle(self):
    slow = self.head
    fast = self.head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    if slow:
        print("Middle node:", slow.data)
    else:
        print("List is empty")

# 5. Delete node by value
def delete(self, value):
    current = self.head
```

```
        if current and current.data == value:
            self.head = current.next
            return

        while current and current.next:
            if current.next.data == value:
                current.next = current.next.next
                return
            current = current.next

        print("Value not found")

    # 6. Reverse linked list
    def reverse(self):
        previous = None
        current = self.head

        while current:
            next_node = current.next
            current.next = previous
            previous = current
            current = next_node

        self.head = previous

    # 7. Sum every two consecutive node values
    def consecutive_sums(self):
        current = self.head

        while current and current.next:
            print(current.data + current.next.data, end=" ")
            current = current.next
        print()

# Example usage
ll = SinglyLinkedList()
ll.create([10, 20, 30, 40, 50])

print("Original list:")
ll.display()

ll.insert_at(25, 2)
```

```
print("After insertion:")
ll.display()

ll.find_middle()

ll.delete(25)
print("After deletion:")
ll.display()

ll.reverse()
print("Reversed list:")
ll.display()

print("Consecutive sums:")
ll.consecutive_sums()
```

